

Visão Geral sobre Arquitetura RISC-V

Foco em design e desempenho

Pedro Tuttman

2025

Sumário

1	Introdução	3
2	Fundamentos de Desempenho em Arquitetura	4
2.1	As Sete Grandes Ideias da Arquitetura de Computadores	4
2.2	O que acontece (por alto) ao executar um programa?	5
3	Conceitos Básicos de Circuitos Lógicos	6
3.1	Portas Lógicas	6
3.2	O Clock	7
3.3	Circuitos Combinacionais e Sequenciais	8
3.4	Flip-Flops e Registradores	8
3.5	Circuitos Integrados (CIs)	8
4	Conceitos Fundamentais da Arquitetura RISC-V	9
4.1	O que é uma ISA?	9
4.2	Filosofia RISC: Instruções Reduzidas	10
4.3	Estrutura Geral da Arquitetura RISC-V	10
4.4	Registradores em RISC-V, ARM e x86-64	12
5	Datapath e Unidade de Controle	13
5.1	Datapath: o caminho dos dados	13
5.2	Unidade de Controle	13
6	Execução com Ciclo Único, Multiciclo e Pipeline	15
6.1	Processador de Ciclo Único	15
6.2	Execução Multiciclo e Pipeline	15
6.3	Comportamento dos registradores intermediários	15
6.4	PC + 4, Branch e Controle de Fluxo	16
6.5	Tipos de Instrução no Pipeline	16
6.6	Exemplos detalhados de execução	16
7	Hazards em Pipeline	17
7.1	Tipos de hazards	17
7.2	Soluções	17
8	Técnicas para Aumentar o Nível de Paralelismo	18
8.1	Aumento da Profundidade do Pipeline	18
8.2	Multiple Issue	18

9	Comparação entre Intel, ARM e RISC-V	19
9.1	Modelo de Instruções (ISA)	19
9.2	Abertura e Licenciamento	19
9.3	Eficiência Energética e Uso Prático	19
9.4	Ecossistema e Suporte de Software	19
9.5	Mercado e Aplicações Atuais	20
10	Memória e Hierarquia de Memórias	21
10.1	Introdução	21
10.2	Princípios de Localidade	21
10.3	Tecnologias de Memória	21
10.4	Memória Cache	22
10.5	Medição de Desempenho	23
10.6	Memória Virtual	23
10.7	Confiabilidade e Tolerância a Falhas	23
10.8	Considerações Finais	23
11	Paralelismo: Do Cliente à Nuvem	24
11.1	Introdução	24
11.2	Níveis de Paralelismo	24
11.3	Memória Compartilhada e Consistência	24
11.4	Clusters e Computação em Nuvem	24
11.5	Arquiteturas Heterogêneas	25
11.6	Desafios de Projetar Sistemas Paralelos	25
11.7	Considerações Finais	25
12	Stack, Buffer Overflow e ROP	26
12.1	Estrutura da Stack	26
12.2	Buffer Overflow	26
12.3	Shellcode Injection	27
12.4	Return Oriented Programming (ROP)	27
12.5	Diferenças entre x86-64 e RISC-V	28
13	Conclusão	29

Capítulo 1

Introdução

Nos últimos anos, a arquitetura RISC-V tem ganhado destaque como uma alternativa aberta, modular e extensível às arquiteturas de processadores proprietárias amplamente utilizadas no mercado, como x86 e ARM. Desenvolvida originalmente na Universidade da Califórnia, em Berkeley, a RISC-V (pronuncia-se “RISC-five”) segue os princípios do *Reduced Instruction Set Computing* (RISC), focando em um conjunto de instruções enxuto e eficiente que permite alto desempenho com simplicidade de implementação.

Um dos grandes diferenciais da RISC-V é seu caráter livre e aberto, o que permite que empresas, instituições de pesquisa e desenvolvedores individuais adotem e modifiquem a arquitetura sem custos de licenciamento. Isso significa que o ISA (*Instruction Set Architecture*) dessa arquitetura está publicamente disponível, permitindo que qualquer pessoa interessada tenha acesso gratuito e irrestrito às informações de como implementar um processador compatível com RISC-V. Essa característica tem impulsionado sua adoção em diversas áreas, desde sistemas embarcados e Internet das Coisas (IoT) até supercomputadores e aplicações educacionais.

Este artigo apresenta uma visão geral da arquitetura RISC-V, explorando seus principais conceitos, vantagens, estrutura modular e aplicações atuais, além de discutir seu impacto potencial no futuro do design de hardware e software.

Capítulo 2

Fundamentos de Desempenho em Arquitetura

Antes de explorar os principais conceitos da arquitetura RISC-V, é fundamental entender os princípios que guiam o desempenho dos computadores modernos. Afinal, toda arquitetura de processador — inclusive a RISC-V — é projetada com o objetivo de otimizar o tempo de execução, reduzir o consumo de energia e aumentar a eficiência no uso dos recursos do sistema.

2.1 As Sete Grandes Ideias da Arquitetura de Computadores

Um dos marcos conceituais mais conhecidos na área são as Sete Grandes Ideias da Arquitetura de Computadores, que orientam o projeto de hardware e software:

- I. Execute a instrução comum rapidamente: otimização dos casos mais frequentes melhora a eficiência geral do sistema.
- II. Melhore o desempenho por meio da paralelização: instruções podem ser executadas simultaneamente em múltiplas unidades de processamento.
- III. Melhore o desempenho por meio do *pipeline*: assim como em uma linha de montagem, diferentes estágios da execução de uma instrução podem ser sobrepostos. Diferente da paralelização, o pipeline foca na execução simultânea de uma linha de comando em um único núcleo, de forma a alavancar a performance de processamento por meio de uma subdivisão da própria instrução em múltiplas etapas. (Esse assunto será abordado mais adiante quando processadores multicíclicos.)
- IV. Hierarquia de memória: usar memórias de diferentes tamanhos e velocidades para equilibrar custo e acesso rápido. (Esse assunto não será abordado nesta edição do artigo.)
- V. Abstrações: uso de camadas de abstração permite melhor produtividade e organização. Esconde a complexidade e permite que o usuário interaja com o sistema sem precisar entender os detalhes técnicos.
- VI. Melhore o desempenho por meio de predições: *Branch Prediction* tenta antecipar qual caminho o código seguirá para evitar atrasos.

- VII. Confiabilidade por meio de redundância: aumenta a confiabilidade, garantindo que, mesmo em casos de falha, ainda haja um recurso alternativo; ou seja, o sistema continuará funcionando.

Entendendo o conceito por trás desses fundamentos, podemos compreender o porquê de cada escolha de implementação, já que essas decisões têm como cerne as ideias acima.

2.2 O que acontece (por alto) ao executar um programa?

Quando você executa um programa, como abrir um navegador ou rodar um código, uma série de processos ocorrem no nível mais baixo do computador.

O código (em C, Python, etc.) é compilado (ou interpretado) em instruções de máquina (assembly), essas instruções são carregadas na memória RAM e a CPU lê essas instruções uma a uma, por meio do ciclo de busca, decodificação e execução, coordenado por um sinal rítmico chamado *clock*. Esse ciclo se repete bilhões de vezes por segundo (GHz), e seu desempenho depende fortemente da organização do processador e da memória.

O clock funciona como um metrônomo interno que dita o ritmo em que as operações do processador ocorrem, garantindo que cada etapa da execução siga uma sequência precisa e sincronizada. Esse sinal periódico é essencial para o funcionamento dos circuitos digitais dentro do processador, sendo gerado por componentes eletrônicos que operam com base em conceitos fundamentais da eletrônica digital, como portas lógicas, circuitos combinacionais e sequenciais, *flip-flops* e circuitos integrados (CIs).

Compreender como esses elementos se organizam e interagem permite uma visão mais clara sobre como arquiteturas como a RISC-V são implementadas fisicamente.

Capítulo 3

Conceitos Básicos de Circuitos Lógicos

Para entender como a CPU executa as instruções e como a arquitetura RISC-V se relaciona com a física dos circuitos, é necessário explorar alguns conceitos fundamentais de circuitos digitais. Esses circuitos são a base para a construção de processadores, como o da RISC-V, e envolvem operações matemáticas e lógicas que ocorrem em alta velocidade, controladas pelo clock.

3.1 Portas Lógicas

A base de qualquer circuito digital são as portas lógicas. Elas realizam operações lógicas fundamentais sobre entradas binárias (0 ou 1), gerando uma saída também binária. As principais portas lógicas são:

- **AND**: a saída é 1 apenas se todas as entradas forem 1;
- **OR**: a saída é 1 se pelo menos uma entrada for 1;
- **NOT**: a saída é o inverso da entrada;
- **XOR**: a saída é 1 se as entradas forem diferentes;
- **NAND**, **NOR** e **XNOR**: inverso das portas AND, OR e XOR, respectivamente.

Essas portas são a “lógica” que possibilita as operações aritméticas e lógicas que a CPU executa ao processar instruções. **Exemplo:** adição de dois bits.

Vamos somar dois números binários de 1 bit: A e B .

Tabela verdade da soma de dois bits

A	B	Soma S / Carry C
0	0	$S = 0, C = 0$
0	1	$S = 1, C = 0$
1	0	$S = 1, C = 0$
1	1	$S = 0, C = 1$

Soma (S): resultado da soma dos dois bits.

Carry (C): se ocorrer um “transbordamento” na soma, um 1 é gerado, que deve ser levado para o próximo bit.

Como construir a soma com portas lógicas? Ao observarmos, a partir da análise da tabela verdade, que o resultado da soma (S) dá 1 quando A e B são diferentes, chegamos à conclusão que S pode ser implementado usando uma porta XOR com entradas A e B : $S = A \oplus B$.

Já para o *carry* (C), como seu resultado será 1 caso *ambas* as entradas sejam 1, concluímos que podemos implementá-lo utilizando uma porta AND: $C = A \wedge B$.

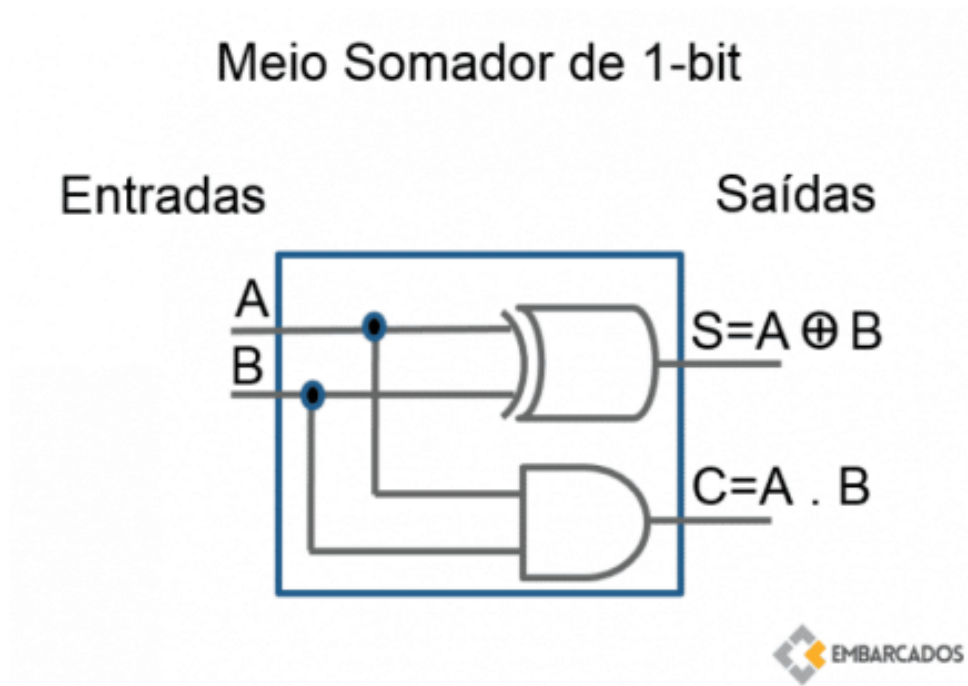


Figura 3.1: Meio Somador

(É chamado de **meio somador** pois não tem como entrada, além dos bits A e B , um carry vindo de outro somador. Ao implementar somadores de mais bits, devemos sobrepor somadores de 1 bit, mas eles devem receber o carry vindo do somador anterior, assim como devemos somar o carry ao realizarmos a conta no papel.)

3.2 O Clock

O clock, como mencionado anteriormente, é o sinal rítmico que sincroniza a execução dos circuitos digitais. Ele garante que todas as operações dentro da CPU aconteçam em uma sequência ordenada, permitindo que múltiplos processos ocorram ao mesmo tempo sem conflitos. O clock é gerado por um oscilador e é crucial para a operação de circuitos sequenciais, como os *flip-flops*.

3.3 Circuitos Combinacionais e Sequenciais

Circuitos Combinacionais: esses circuitos geram saídas diretamente baseadas nas entradas atuais, sem considerar o histórico de entradas anteriores. Exemplo: somadores e multiplicadores. A saída é uma função combinada das entradas em tempo real.

Circuitos Sequenciais: ao contrário dos combinacionais, esses circuitos têm a capacidade de lembrar o estado anterior, armazenando informações entre as mudanças de clock. Isso é possível por meio de *flip-flops*, que atuam como unidades de memória pequenas. Esses circuitos são fundamentais para armazenar o estado da execução de um programa e para o funcionamento dos registradores e *caches* na CPU.

3.4 Flip-Flops e Registradores

Os *flip-flops* são circuitos lógicos sequenciais simples que armazenam um bit de informação. Eles são ativados por um sinal de clock e mantêm o valor até a chegada do próximo sinal de clock. Vários *flip-flops* podem ser agrupados para formar **registradores**, que são usados para armazenar dados temporários dentro da CPU, como instruções em execução ou valores intermediários de cálculos.

3.5 Circuitos Integrados (CIs)

Os circuitos integrados (CIs) são conjuntos de componentes eletrônicos, como transistores, resistores e capacitores, que são integrados em uma única peça de silício. Eles são fundamentais para construir processadores, já que concentram as funções de múltiplos componentes em um único chip, otimizando o espaço e o desempenho do sistema.

Esses conceitos formam a base do funcionamento de qualquer processador. Agora que compreendemos como os circuitos digitais funcionam, é possível analisar com mais profundidade as implementações físicas da arquitetura RISC-V e como ela utiliza esses princípios para otimizar o desempenho e a eficiência.

Capítulo 4

Conceitos Fundamentais da Arquitetura RISC-V

Com os fundamentos da lógica digital e do funcionamento interno da CPU bem estabelecidos, podemos agora abordar a arquitetura RISC-V com mais propriedade. Neste capítulo, focaremos nos conceitos centrais que definem como essa arquitetura opera em nível de projeto e execução.

4.1 O que é uma ISA?

Uma *Instruction Set Architecture* (ISA) — ou Arquitetura de Conjunto de Instruções — é a interface entre o software e o hardware de um sistema computacional. Trata-se de uma especificação formal que descreve:

- Quais instruções o processador é capaz de executar (como somar, comparar, carregar dados da memória, etc.);
- Como essas instruções são codificadas em bits;
- Quantos registradores estão disponíveis e como são acessados;
- Quais modos de endereçamento de memória são suportados;
- E, por fim, quais os efeitos de cada instrução sobre o estado do processador (por exemplo, quais registradores ou sinais são modificados).

A ISA é, portanto, a “linguagem nativa” do processador. Quando um programa em C, Python ou qualquer outra linguagem é compilado, o compilador gera código de máquina que obedece às regras da ISA do processador-alvo.

Por que isso importa? Porque é a ISA que permite que software seja portátil (desde que compile para a ISA certa), e que hardware seja substituído ou atualizado sem que o sistema como um todo precise ser reescrito. No caso do RISC-V, essa portabilidade vai além: por ser uma arquitetura livre e aberta, qualquer empresa, universidade ou pessoa pode desenvolver um processador compatível com a ISA RISC-V, sem pagar royalties.

Além disso, por ser modular e bem documentada, a ISA RISC-V permite adaptações para diferentes cenários: desde microcontroladores embarcados até processadores complexos de 64 ou 128 bits.

4.2 Filosofia RISC: Instruções Reduzidas

A arquitetura RISC-V segue a filosofia RISC (*Reduced Instruction Set Computer*), um paradigma de design de processadores que visa simplificar o conjunto de instruções para obter maior eficiência e previsibilidade.

Princípios do RISC:

- Instruções simples e de tamanho fixo (geralmente 32 bits);
- Execução em um único ciclo de clock, sempre que possível;
- Acesso à memória somente por instruções de carga (*load*) e armazenamento (*store*);
- Grande número de registradores, reduzindo a necessidade de acessar a memória frequentemente;
- Pipeline eficiente e previsível;
- Separação clara entre instruções de cálculo, controle de fluxo e acesso à memória.

Ao invés de criar instruções complexas que fazem múltiplas operações ao mesmo tempo (como em arquiteturas CISC, como x86), o RISC opta por dividir tarefas em instruções atômicas e rápidas. Isso pode exigir mais instruções para realizar uma tarefa, mas resulta em maior paralelismo, menor consumo de energia e maior facilidade de implementação em hardware.

Essa simplicidade torna o RISC-V não apenas mais eficiente, mas também mais adequado ao ensino, pesquisa e inovação, pois permite entender e modificar a arquitetura com mais clareza.

4.3 Estrutura Geral da Arquitetura RISC-V

O núcleo da ISA RISC-V é o conjunto RV32I, ou seja, a versão de 32 bits com instruções inteiras (*Integer*). Essa base é obrigatória, e todas as implementações da arquitetura devem suportá-la. A partir dela, é possível adicionar extensões modulares, como multiplicação (M), ponto flutuante (F, D), instruções atômicas (A), e outras.

a) Registradores

O RV32I define 32 registradores de 32 bits, nomeados de x0 a x31. Os registradores são o tipo de memória mais rápido dentro de um computador, e configuram-se, basicamente, como um conjunto de *flip-flops* tipo D síncronos (compartilham o mesmo clock). Além da notação numérica, existe uma convenção de nomes que indica o uso típico de cada registrador.

Todos os registradores, exceto o x0, podem ser usados livremente em instruções aritméticas, lógicas e de controle. No entanto, mesmo que seja tecnicamente possível usar qualquer registrador (exceto o x0, que sempre contém o valor zero), a arquitetura RISC-V adota convenções de uso para os registradores, especialmente para facilitar a interoperabilidade entre funções, bibliotecas e compiladores. Essas convenções são chamadas de *Calling Convention* e ajudam a organizar o uso dos registradores em blocos com finalidades específicas.

Convenções de uso dos registradores RISC-V:

Registrador constante (x0 – zero)

- x0 / zero: sempre contém o valor 0.
- Útil para operações como `add x5, x6, x0` (copia x6 em x5). Mesmo que usar `addi x5, x6, 0`. A diferença é o tipo de instrução (R-type vs I-type).
- Não pode ser modificado.

Retorno e controle de fluxo — x1/ra (return address)

- Guarda o endereço de retorno de chamadas de função;
- Ao usar `jal` ou `jalr`, o endereço da próxima instrução vai para `ra`.

Instruções relacionadas:

- `jal ra, funcao`: salta para `funcao` e armazena o endereço de retorno em `ra`.
- `jalr ra, 0(x5)`: salta para o endereço em `x5`; também armazena o retorno em `ra`.
- `ret` (pseudoinstrução): abrevia `jalr x0, 0(ra)`.

Analogia com o `rip` (x86-64): em x86, o endereço de retorno é empilhado; no RISC-V ele vai para `ra`. Se a função chamada fizer novas chamadas, salve `ra` na pilha para não sobrescrever.

Temporários — x5/x6/x7 (t0–t2), x28–x31 (t3–t6) Registradores temporários não preservados entre chamadas de função; a função pode usá-los livremente sem restaurar.

Pilha e controle de quadro de ativação

- `x2 / sp` (*Stack Pointer*): aponta para o topo da pilha.
- `x8 / s0` ou `fp`, `x9 / s1`, `x18–x27 / s2–s11`: *saved registers* — quem usa deve salvar/restaurar.

Passagem de argumentos e retorno de funções — x10–x17 / a0–a7

- `a0–a1`: retorno de funções.
- `a0–a7`: passagem de argumentos (até 8). Argumentos extras são passados pela pilha (abaixo de `sp`).

Temporários adicionais

- `x3 / gp` (*Global Pointer*): aponta para área de dados globais.
- `x4 / tp` (*Thread Pointer*): usado em sistemas com múltiplas *threads*.

Por que seguir as convenções? Elas tornam o código modular, reutilizável e interoperável; compiladores e *linkers* dependem delas para gerar código correto.

4.4 Registradores em RISC-V, ARM e x86-64

Nas arquiteturas RISC, como RISC-V e ARM, todas as operações aritméticas são realizadas apenas entre registradores, sendo que o acesso à memória é feito exclusivamente por meio de instruções de load/store. Esse modelo exige um conjunto maior de registradores de propósito geral, normalmente 32, para permitir que o processador mantenha mais valores prontos para uso imediato, reduzindo acessos à RAM. Já no x86-64, uma arquitetura CISC, as instruções podem operar diretamente sobre a memória (por exemplo, somar um valor de um registrador com outro armazenado na RAM). Isso reduz a necessidade de tantos registradores de propósito geral, motivo pelo qual essa arquitetura oferece apenas 16.

b) Instruções e seus formatos

No RISC-V, cada instrução ocupa exatamente 32 bits, o que facilita sua decodificação e alinhamento na memória. As instruções são classificadas em formatos, de acordo com sua função. Cada campo tem um papel específico:

Tipo	Uso Principal	Campos (ordem binária: [31 → 0])
R-type	Operações entre registradores	funct7, rs2, rs1, funct3, rd, opcode
I-type	Cálculos com imediato, loads, jalr	imm[11:0], rs1, funct3, rd, opcode
S-type	Armazenamento (store)	imm[11:5], rs2, rs1, funct3, imm[4:0], opcode
B-type	Desvios condicionais (beq, bne, ...)	imm[12], imm[10:5], rs2, rs1, funct3, imm[4:1], imm[11], opcode
U-type	Carga de constantes em 20 bits (lui, auipc)	imm[31:12], rd, opcode
J-type	Salto longos (jal)	imm[20], imm[10:1], imm[11], imm[19:12], rd, opcode

Figura 4.1: Tabela dos tipos de instrução em RISC-V

- **opcode**: identifica a operação principal;
- **rd**: registrador de destino;
- **rs1**, **rs2**: registradores de origem;
- **imm**: valor imediato (número embutido na instrução);
- **funct3**, **funct7**: subcódigos que ajudam a diferenciar instruções com o mesmo opcode.

Esse design modular e padronizado permite implementar decodificadores de instrução simples e eficientes. Como todos os tipos de instrução terminam com o *opcode*, um jeito simples de identificar uma instrução em binário é isolar os 7 bits menos significativos, detectar o *opcode* e, a partir disso, dividir os campos corretamente.

Capítulo 5

Datapath e Unidade de Controle

Para entendermos o funcionamento interno de um processador, precisamos conhecer seus dois componentes fundamentais: o **Datapath** (caminho de dados) e a **Unidade de Controle** (*Control Unit*). Ambos trabalham em conjunto para realizar as instruções descritas no conjunto de instruções da arquitetura (como o RISC-V), de forma sincronizada com o clock do sistema.

5.1 Datapath: o caminho dos dados

O Datapath é composto pelos componentes físicos que realmente executam as instruções. Entre os principais elementos estão:

- **Program Counter (PC)**: registrador que guarda o endereço da próxima instrução a ser buscada (análogo o `rip` no `x86_64`);
- **Memória de Instruções**: armazena o programa a ser executado;
- **Register File**: banco de registradores (`x0` a `x31`), com duas leituras e uma escrita por ciclo;
- **ULA**: unidade lógica e aritmética que executa operações aritméticas e lógicas;
- **MUXes**: selecionam entre múltiplas entradas, conforme sinais de controle;
- **Extensor de Imediatos**: interpreta o campo de imediato e o converte para 32 bits com sinal;
- **Memória de Dados**: utilizada para *load* e *store*.

Harvard vs Von Neumann Arquitetura Harvard usa memórias separadas para instruções e dados, permitindo acessos simultâneos; Von Neumann usa memória única, é mais simples, porém exige cuidado com acessos simultâneos.

5.2 Unidade de Controle

Responsável por gerar os sinais de controle que determinam o comportamento do datapath a cada ciclo. Em termos de circuitos, pode ser vista como uma máquina de estados finitos (Mealy ou Moore). A unidade de controle interpreta *opcode* e *funct3/funct7* e gera sinais

como RegWrite, MemRead, MemWrite, ALUSrc, MemToReg, Branch, ALUOp, que orquestram o datapath.

Capítulo 6

Execução com Ciclo Único, Multiciclo e Pipeline

6.1 Processador de Ciclo Único

No modelo de ciclo único, cada instrução é executada inteiramente em um único ciclo de clock. Problemas: o tempo do clock precisa ser igual ao tempo da instrução mais lenta (ex.: *load*). Instruções simples ficam penalizadas.

6.2 Execução Multiciclo e Pipeline

a) Multiciclo

Cada instrução é dividida em múltiplos ciclos, reutilizando componentes. O tempo de clock é ajustado à etapa mais lenta.

b) Pipeline

Aproveita o paralelismo entre instruções. Modelo clássico de 5 estágios:

1. IF (Instruction Fetch): busca da memória;
2. ID (Instruction Decode): decodifica, lê registradores, estende imediato;
3. EX (Execute): ULA e avaliação de *branch*;
4. MEM (Memory): acessos à memória;
5. WB (Write Back): escrita no registrador.

6.3 Comportamento dos registradores intermediários

Registradores especiais entre etapas: **IF/ID**, **ID/EX**, **EX/MEM**, **MEM/WB**. Armazenam dados e sinais de controle dos estágios.

6.4 PC + 4, Branch e Controle de Fluxo

Em cada ciclo, o PC normalmente é incrementado por 4. Para desvios, o novo PC depende do imediato e do resultado da comparação: se **Branch** for ativado e a condição satisfeita, novo PC = PC + *offset*; caso contrário, PC segue com PC + 4.

6.5 Tipos de Instrução no Pipeline

R-type: usa registradores; resultado vai para rd.

I-type: imediato é estendido e vai para a ULA via ALUSrc; pode acessar memória (load).

B-type: imediato somado ao PC; ULA compara rs1 e rs2; desvio decidido em EX.

6.6 Exemplos detalhados de execução

Ex.: add x5, x1, x2

1. IF: busca; $PC \leftarrow PC + 4$; registra em IF/ID.
2. ID: decodifica; lê x1 e x2; gera sinais: RegWrite=1, ALUSrc=0, ALUOp=10, MemRead=0, MemWrite=0, MemToReg=0, Branch=0; armazena em ID/EX.
3. EX: ULA soma x1 e x2; resultado para EX/MEM (RegWrite=1, MemToReg=0).
4. MEM: sem acesso à memória; passa para MEM/WB.
5. WB: escreve em x5.

Ex.: lw x6, 12(x1)

1. IF: busca; $PC \leftarrow PC + 4$; guarda em IF/ID.
2. ID: lê x1 e extrai imediato 12; sinais: RegWrite=1, ALUSrc=1, ALUOp=00, MemRead=1, MemWrite=0, MemToReg=1, Branch=0.
3. EX: ULA calcula endereço efetivo: $x1 + 12$; armazena em EX/MEM (MemRead=1, MemToReg=1).
4. MEM: lê da memória; dado para MEM/WB.
5. WB: escreve em x6.

Considerações adicionais

- **Hazards (perigos):** dependências entre instruções (RAW, WAR, WAW).
- **Registradores Intermediários:** mantêm resultados parciais e sinais de controle.
- **Estado da máquina:** conteúdo atual de PC, registradores, memórias, registradores intermediários, sinais e resultados parciais da ULA.

Capítulo 7

Hazards em Pipeline

A sobreposição de instruções no pipeline aumenta o desempenho, mas introduz conflitos e dependências (*hazards*).

7.1 Tipos de hazards

Structural Hazards

Conflitos de recurso (ex.: memória única em Von Neumann).

Data Hazards

Quando uma instrução depende de um dado ainda não produzido.

- **RAW (Read After Write)**: B lê registrador que A irá escrever.
- **WAR (Write After Read)** e **WAW (Write After Write)**: comuns fora de ordem.

Control Hazards

Desvios (*branch/jump*), destino de PC conhecido apenas após comparação.

7.2 Soluções

Structural

Duplicação de recursos (Harvard) ou *interlocking* (stalls).

Data

Forwarding e *stalling*.

Control

Stalling, *delayed branch*, *branch prediction* (estática/dinâmica: 1-bit, 2-bit saturating, correlating, *tournament*).

Capítulo 8

Técnicas para Aumentar o Nível de Paralelismo

8.1 Aumento da Profundidade do Pipeline

Dividir etapas em subetapas menores reduz o tempo de cada ciclo e permite frequências maiores. Atenção a gargalos (etapas mais lentas impõem limite do clock). Exemplos: subdividir EX (EX1/EX2) ou IF (IF1/IF2).

8.2 Multiple Issue

Emitir mais de uma instrução por ciclo. Duas abordagens:

- **Superescalar** (dinâmica): hardware decide paralelismo em tempo de execução.
- **VLIW** (estática): compilador agrupa instruções em *issue slots*.

Throughput Aumenta a quantidade de instruções completadas por unidade de tempo (mesmo com latência similar).

Especulação Executa instruções de forma antecipada; distinta de predição, mas dela dependente.

Observação sobre vulnerabilidades *Spectre* e *Meltdown* exploram efeitos colaterais de execução especulativa (*side-channels* via cache).

Capítulo 9

Comparação entre Intel, ARM e RISC-V

9.1 Modelo de Instruções (ISA)

- **Intel (x86/x86-64):** CISC, grande número de instruções, alta complexidade de decodificação.
- **ARM:** RISC, conjunto enxuto; extensões modernas (Thumb, NEON) adicionam complexidade.
- **RISC-V:** RISC, modular e aberto.

9.2 Abertura e Licenciamento

- **Intel:** proprietária; sem acesso aberto ao ISA.
- **ARM:** licenciada; empresas pagam para usar/desenvolver.
- **RISC-V:** gratuita e aberta; sem royalties.

9.3 Eficiência Energética e Uso Prático

- **Intel:** foco em alto desempenho; maior consumo energético (PCs/servidores).
- **ARM:** eficiência energética; dominante em móveis/IoT; expansão para laptops/servidores.
- **RISC-V:** modularidade permite adaptar de IoT a sistemas potentes; maturação em alto desempenho em curso.

9.4 Ecossistema e Suporte de Software

- **Intel:** ecossistema mais maduro; suporte amplo.
- **ARM:** bem estabelecido (embarcados/Android); GCC/LLVM/LINUX.

- **RISC-V**: em crescimento; já suportado por GCC/LLVM e distros Linux; softwares proprietários e legados ainda em evolução.

9.5 Mercado e Aplicações Atuais

- **Intel**: PCs, servidores, workstations.
- **ARM**: smartphones, tablets, embarcados; expansão para servidores/laptops (ex.: Apple Silicon).
- **RISC-V**: embarcados, acadêmicos, aplicações customizadas; investimentos de empresas como SiFive, Alibaba, NVIDIA.

Capítulo 10

Memória e Hierarquia de Memórias

10.1 Introdução

O avanço dos processadores trouxe um aumento significativo no desempenho da CPU, mas a memória principal não acompanhou esse crescimento no mesmo ritmo. Esse descompasso entre velocidade do processador e latência da memória é chamado de *memory wall*. Sem estratégias de hierarquia de memória, os processadores modernos passariam grande parte do tempo ociosos, aguardando dados da memória. A solução arquitetural é a **hierarquia de memórias**, que combina diferentes níveis (registradores, caches, memória principal, armazenamento secundário), explorando um equilíbrio entre custo, desempenho e capacidade.

10.2 Princípios de Localidade

Dois princípios guiam o projeto da hierarquia de memória:

- **Localidade temporal:** dados acessados recentemente têm grande chance de serem acessados novamente em breve. Exemplo: variáveis em laços de repetição.
- **Localidade espacial:** se um endereço é acessado, vizinhos próximos tendem a ser acessados também. Exemplo: percorrer um array sequencialmente.

Exemplo prático em C:

Listing 10.1: Acesso com localidade espacial

```
for (int i = 0; i < N; i++) {  
    soma += vetor[i];    // Acesso sequencial – alta localidade espacial  
}
```

Se o vetor for percorrido com saltos (por exemplo, acessando apenas índices múltiplos de 100), a localidade espacial diminui e a performance cai.

10.3 Tecnologias de Memória

Cada nível da hierarquia é implementado com uma tecnologia distinta:

- **Registradores:** extremamente rápidos, integrados à CPU, mas escassos e caros.

- **SRAM** (Static RAM): utilizada em caches, muito rápida mas com baixo fator de densidade e alto custo.
- **DRAM** (Dynamic RAM): usada na memória principal, mais lenta que SRAM, mas com maior densidade e menor custo.
- **Armazenamento secundário**: HDs e SSDs, com grande capacidade, mas latência muito maior.

Tabela 10.1: Comparação entre tecnologias de memória

Tecnologia	Latência típica	Capacidade	Custo/bit
Registradores	< 1 ns	Bytes	Muito alto
SRAM (cache)	1–10 ns	KB–MB	Alto
DRAM (RAM)	50–100 ns	GBs	Médio
SSD/HDD	μ s–ms	TBs	Baixo

10.4 Memória Cache

A cache é a ponte entre CPU e memória principal. Armazena blocos de dados acessados recentemente, explorando localidade.

Organização

- **Mapeamento direto**: cada bloco de memória só pode ocupar uma posição específica na cache.
- **Associativa por conjunto**: cada bloco pode ocupar uma entre várias linhas em um conjunto.
- **Associativa total**: qualquer bloco pode ser armazenado em qualquer linha.

Políticas de substituição

Quando ocorre um miss e é necessário carregar um novo bloco:

- **LRU (Least Recently Used)**: substitui o bloco não usado há mais tempo.
- **FIFO**: substitui o mais antigo.
- **Aleatório**: escolhe aleatoriamente um bloco para remoção.

Políticas de escrita

- **Write-through**: cada escrita vai simultaneamente para a cache e memória principal (consistente, mas mais lento).
- **Write-back**: a escrita é feita na cache, e só é propagada para a memória quando o bloco é substituído (mais rápido, mas exige controle).

10.5 Medição de Desempenho

O desempenho da hierarquia de memória é medido pelo tempo médio de acesso (AMAT):

$$AMAT = Hit\ Time + Miss\ Rate \times Miss\ Penalty \quad (10.1)$$

Exemplo: cache com tempo de hit de 1 ns, taxa de miss de 5% e penalidade de 50 ns:

$$AMAT = 1 + 0.05 \times 50 = 3.5\ ns \quad (10.2)$$

10.6 Memória Virtual

A memória virtual abstrai o espaço físico de endereços, dando a cada processo a ilusão de possuir todo o espaço de endereçamento disponível.

- Divisão em **páginas** (típico: 4 KB).
- Tradução de endereços via **tabela de páginas**.
- Uso de **TLB** (Translation Lookaside Buffer) para acelerar traduções.
- Fornece isolamento, proteção e compartilhamento entre processos.

10.7 Confiabilidade e Tolerância a Falhas

Em sistemas críticos, é necessário proteger a memória contra falhas:

- **Paridade:** detecta erros de um bit.
- **ECC (Error-Correcting Code):** detecta e corrige erros de 1 bit, detecta múltiplos bits.
- **RAID:** técnicas de redundância aplicadas a discos, aumentando confiabilidade e disponibilidade.

10.8 Considerações Finais

A hierarquia de memória é essencial para reduzir o impacto do descompasso entre CPU e memória. Combinando localidade, caches, memória virtual e técnicas de confiabilidade, sistemas modernos conseguem equilibrar desempenho, custo e escalabilidade.

Capítulo 11

Paralelismo: Do Cliente à Nuvem

11.1 Introdução

O aumento da frequência de clock encontrou limites físicos e de dissipação térmica. Assim, a indústria migrou para arquiteturas **multicore**, explorando paralelismo. Hoje, o paralelismo é explorado desde o nível de instruções até grandes data centers em nuvem.

11.2 Níveis de Paralelismo

- **Paralelismo em nível de instrução (ILP)**: técnicas como pipeline, execução fora de ordem e múltipla emissão.
- **Paralelismo em nível de dados (DLP)**: operações vetoriais e SIMD, aproveitando padrões em arrays e matrizes.
- **Paralelismo em nível de threads (TLP)**: múltiplos núcleos ou hardware multithreading (SMT).
- **Paralelismo em nível de tarefas**: execução de processos independentes em clusters e data centers.

11.3 Memória Compartilhada e Consistência

O uso de múltiplos núcleos exige garantir consistência entre caches.

- **Problema da coerência de cache**: quando dois núcleos têm cópias diferentes da mesma linha.
- **Protocolos de coerência (MESI)**: garantem atualização consistente entre caches.
- **Modelos de consistência**: sequential consistency (mais intuitivo, mas restritivo) vs. relaxed models (maior desempenho, mas mais complexos).

11.4 Clusters e Computação em Nuvem

O paralelismo vai além do chip:

- **Clusters:** computadores interligados cooperando em tarefas distribuídas.
- **Data centers:** milhares de servidores, base da computação em nuvem.
- **Exemplo:** uso de MapReduce e frameworks como Hadoop e Spark.

11.5 Arquiteturas Heterogêneas

Muitos sistemas combinam diferentes tipos de processadores:

- **CPUs + GPUs:** CPUs para controle e tarefas gerais; GPUs para alto paralelismo (ex.: aprendizado de máquina).
- **Aceleradores especializados:** para IA, criptografia, redes neurais.

11.6 Desafios de Projetar Sistemas Paralelos

- Comunicação entre núcleos e latência de memória compartilhada.
- Balanceamento de carga entre múltiplas threads/tarefas.
- Escalabilidade: manter desempenho ao aumentar o número de núcleos/máquinas.

11.7 Considerações Finais

O futuro da computação está diretamente ligado ao paralelismo, desde dispositivos pessoais até nuvens globais. A combinação de ILP, DLP, TLP e arquiteturas heterogêneas permite extrair desempenho em diferentes níveis. Entretanto, os desafios de coerência, comunicação e escalabilidade continuam sendo áreas centrais de pesquisa e desenvolvimento.

Capítulo 12

Stack, Buffer Overflow e ROP

12.1 Estrutura da Stack

A memória de um processo é organizada em regiões distintas: o segmento de código, o segmento de dados, a heap e a stack. A **stack** é usada para chamadas de função, crescendo em direção contrária à heap. Cada chamada de função cria um **stack frame**, que contém:

- o endereço de retorno (para onde a CPU deve saltar ao final da função),
- o frame pointer anterior,
- as variáveis locais da função.

Quando a função termina, esse espaço é liberado automaticamente. O funcionamento em pilha (LIFO — *last in, first out*) garante que a função mais recente seja sempre a primeira a ser finalizada, retomando o controle da execução para a função chamadora.

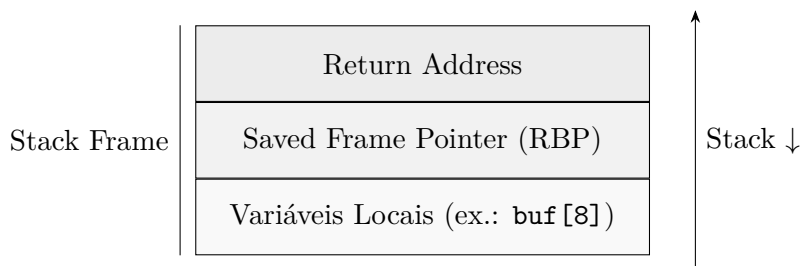


Figura 12.1: Estrutura simplificada de um *stack frame* e direção de crescimento da stack.

12.2 Buffer Overflow

Um **buffer overflow** ocorre quando dados armazenados em uma variável local excedem o espaço reservado para ela. Esse excesso pode sobrescrever áreas adjacentes da stack, incluindo o endereço de retorno do stack frame. Com isso, o fluxo de execução do programa pode ser redirecionado para regiões indesejadas. Historicamente, essa vulnerabilidade foi um dos principais vetores de ataque em programas escritos em C, onde funções como `strcpy` não verificam limites de tamanho de entrada.

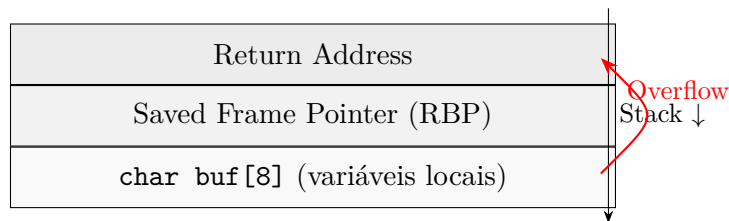


Figura 12.2: Buffer overflow: dados excedem `buf` e alcançam o *Return Address*.

12.3 Shellcode Injection

Em ataques clássicos, o atacante colocava um pequeno trecho de código em linguagem de máquina — chamado **shellcode** — diretamente dentro do buffer. Esse código normalmente chamava o interpretador de comandos do sistema operacional (por exemplo, `/bin/sh` no Unix), abrindo uma shell para controle do sistema. O endereço de retorno era sobrescrito para apontar para o início do buffer, fazendo com que, ao encerrar a função, a CPU começasse a executar o shellcode injetado. Com a introdução de proteções modernas como NX/DEP (Non-Executable/ Data Execution Prevention), que marcam a stack e a heap como não executáveis, essa técnica tornou-se mais difícil de aplicar.

12.4 Return Oriented Programming (ROP)

Para contornar as restrições de execução, surgiu o **Return Oriented Programming (ROP)**. Nessa técnica, em vez de injetar código novo, o atacante **reutiliza trechos de código já existentes** no binário ou em bibliotecas carregadas, conhecidos como **gadgets**. Cada gadget termina em uma instrução de retorno (**ret**) e realiza uma operação simples (como mover valores para registradores ou efetuar uma soma). Ao encadear vários gadgets pela manipulação da stack, é possível construir uma sequência de instruções que executa ações arbitrárias. Um caso especial é o **ret2libc**, onde o atacante manipula o fluxo para chamar diretamente funções da biblioteca padrão C, como `system("/bin/sh")`.

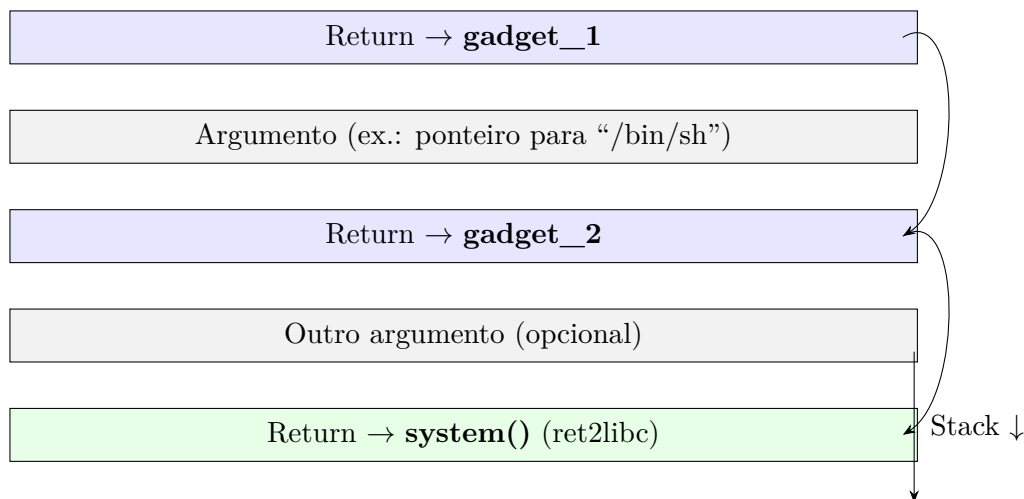


Figura 12.3: Cadeia ROP: gadgets existentes são encadeados até chamar `system("/bin/sh")`.

12.5 Diferenças entre x86-64 e RISC-V

Embora os princípios de buffer overflow, shellcode e ROP sejam gerais, a exploração varia conforme a arquitetura:

- **x86-64:** O endereço de retorno é sempre salvo diretamente na stack após uma instrução `call`. A arquitetura CISC oferece instruções complexas, resultando em uma grande quantidade e diversidade de gadgets, o que facilita a criação de cadeias ROP.
- **RISC-V:** O endereço de retorno inicialmente fica no registrador `ra` (x1) e só é salvo na stack caso o prólogo da função o faça explicitamente. Isso altera a dinâmica dos ataques, pois o invasor precisa lidar tanto com o registrador quanto com a stack. Além disso, por ser uma arquitetura RISC, as instruções são mais simples e homogêneas, reduzindo a quantidade e variedade de gadgets disponíveis para ROP.

Assim, embora o conceito seja universal, os detalhes da execução de ataques de buffer overflow e ROP diferem substancialmente entre x86-64 e RISC-V.

Capítulo 13

Conclusão

Exploramos fundamentos das arquiteturas RISC, aplicações práticas, técnicas modernas de paralelismo e, por fim, comparamos as principais arquiteturas da atualidade (Intel, ARM e RISC-V). Foi possível observar como cada arquitetura evoluiu para atender diferentes demandas do mercado, seja em desempenho, eficiência energética, flexibilidade ou abertura tecnológica.

A arquitetura x86 continua dominante em alto desempenho e compatibilidade com softwares legados, enquanto ARM lidera em dispositivos móveis e sistemas embarcados, graças à eficiência energética e a um vasto ecossistema. Por outro lado, o RISC-V surge como alternativa promissora, oferecendo liberdade de design, custo reduzido e potencial de inovação, especialmente em contextos de pesquisa, educação e aplicações customizadas.

Além disso, discutimos como técnicas como o aumento da profundidade do pipeline e o *multiple issue* contribuem para elevar o desempenho dos processadores modernos, enfrentando os desafios da execução paralela e da exploração do *throughput* computacional.

Referências

- Livro Texto: *Computer Organization and Design* (RISC-V Edition).
- Aulas ministradas pelo professor UFRJ/COPPE Diego Dutra.